Inheritance is useful for creating common base files and reusing structure throughout your web pages, but you may want to have a chunk of HTML repeated within your page instead. A nice companion to inheritance is the ability to include other templates inside your HTML.

3.8.2 Including an HTML snippet

I heard you want to have templates in your templates. Well, there is a tag for that. The `{% include %}` tag loads a subtemplate into your template. There are two common uses for this: organizing your template files and reusable components. HTML files have a tendency to get overly long. If you want to organize them in pieces, you can compose a page from a series of included subtemplates. The simplest use of `{% include %}` takes a single argument: the name of the template to be included. The following listing shows a sample file that includes two subtemplates.

Listing 3.17 An HTML file that includes two subtemplates

```
<!-- Sample template using include -->
{% extends "base.html" %}

{% block content %}

    <h1>Nicky's Blog</h1>

    {% include "blog1.html %}

    {% include "musicians_list.html" %}

{% endblock content %}
```

Once you have subtemplates, you can include them in multiple files, giving you reusable components. This is similar to a function call in your code: you encapsulate some HTML together and then compose it into other templates as needed. Like a function, you can even declare arguments to your subtemplate. The `{% include %}` tag has an alternate form that uses the keyword `with`. This form gives you the ability to declare key–value pairs as context in the subtemplate.

Consider a case in which you are using a third-party comment section, such as Disqus, on your website or including page view tracking JavaScript. In both cases, you need to include some HTML that is parameterized for the given page. You can build this in a reusable fashion by creating a subtemplate with a variable and then use the `with` clause when instantiating the `{% include %}` tag.

The following is a subtemplate that is part of a pagination footer. The view rendering the template takes a query string specifying a page number. You'll learn how to write this kind of view in a later chapter. For now, imagine a view that renders different content based on a query parameter named `page`. Each instance of the subtemplate contains links to the previous and next page. The subtemplate in listing 3.18 has two links, based on variables named `prev` and `next`. All of this is wrapped in `{% if %}` tags;